



Polymarket: CCIP Bridge-Router v2 diff

Security Review

Cantina Managed review by:

Giovanni Di Siena, Lead Security Researcher

Red Swan, Security Researcher

July 13, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Non-canonical <code>bytes32</code> recipient passes source validation but causes permanent asset loss on destination	4
3.1.2	OO-backed atomic neg-risk markets cannot truthfully settle the synthetic Other outcome	4
3.2	Low Risk	5
3.2.1	Synthetic NegRisk fallback resolution bypasses per-condition pause	5
3.2.2	Oracle Pause Consumes Dispute Window Liveness	6
3.2.3	CcipBridge Cannot Interact With Non-EVM Peers	7
3.2.4	Callers can choose CCIP ordering policy for the shared bridge sender lane	7
3.2.5	<code>bridgePositions()</code> does not enforce a source-side batch size cap	8
3.3	Gas Optimization	9
3.3.1	Event operations accumulate dead memory across outcome iterations	9
3.4	Informational	10
3.4.1	Event-helper names overstate their NegRisk-only checks	10
3.4.2	<code>NegRiskModule</code> & <code>CombinatorialModule</code> Exceed EIP-170 24KB Limit	10
3.4.3	Peer rotation requires a bridge-wide send pause	11
3.4.4	Incorrect <code>BridgeBase</code> storage gap size shifts derived <code>peers</code> storage slot in <code>CcipBridge</code>	12

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Jun 24th to Jul 2nd the Cantina team conducted a review of [polymarket-v2](#) on commit hash [264fcb43](#). The team identified a total of **12** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	1	1
Low Risk	5	2	3
Gas Optimizations	1	1	0
Informational	4	2	2
Total	12	6	6

2.1 Scope

The security review had the following components in scope for [polymarket-v2](#) on commit hash [264fcb43](#):

```
src
├── bridge
│   ├── CcipBridge.sol
│   └── abstract
│       └── BridgeBase.sol
├── libraries
│   ├── BridgePayloads.sol
│   ├── CrossChainTypes.sol
│   └── Ids.sol
├── modules
│   ├── BinaryModule.sol
│   ├── CombinatorialModule.sol
│   ├── NegRiskModule.sol
│   └── abstract
│       ├── BaseModule.sol
│       ├── ModuleErrors.sol
│       └── OracleModule.sol
│   └── migration
│       ├── BinaryMigrationMixin.sol
│       └── NegRiskMigrationMixin.sol
├── oracle
│   ├── OracleAggregator.sol
│   └── abstract
│       └── OracleModuleBase.sol
│   ├── mixins
│   │   └── Auth.sol
│   └── modules
│       └── reporters
│           └── EOAREporterModule.sol
└── routers
    └── Router.sol
```

3 Findings

3.1 Medium Risk

3.1.1 Non-canonical `bytes32` recipient passes source validation but causes permanent asset loss on destination

Severity: Medium Risk

Context: `BridgeBase.sol#L178-L196`, `BridgeBase.sol#L224-L235`

Description: `BridgeBase.bridgePositions` and `BridgeBase.bridgeCollateral` accept a `bytes32_recipient` parameter and validate only that it is non-zero.

The destination handlers however decode it as `address`. Solidity's strict ABI decoder requires the upper 96 bits of an `address` slot to be zero. If a caller supplies a `bytes32` recipient with any of the upper bits set, the source-side burn succeeds and is irreversible, but the destination `abi.decode` reverts, the CCIP message cannot be retried with a corrected recipient, and the assets become permanently lost.

`BridgeRouter.bridgePositionsTo` and `BridgeRouter.bridgeCollateralTo` are the primary exposed entry points — they accept arbitrary `bytes32_recipient` from any caller and forward it unchanged. The self-send variants (`bridgePositions` , `bridgeCollateral`) are not affected because they construct the recipient via `bytes32(uint256(uint160(msg.sender)))`, which is always canonical.

Destination chains are owner-whitelisted via the `peers` mapping, so the bug can only be triggered on explicitly configured routes. This limits likelihood but does not eliminate it — any caller on a live route is a potential victim.

Recommendation: The appropriate fix depends on whether non-EVM destination support is intended.

If `CcipBridge` is intended to remain EVM-only (which the current implementation implies, since `peers` is typed as `address` and cannot store a non-EVM peer), change `_recipient` from `bytes32` to `address` throughout the send path (`IBridge` , `BridgeBase` , `BridgeRouter` , `BridgePayloads`). The compiler enforces canonicity at every call site with no runtime overhead and no new error needed.

If non-EVM destination support is a future goal, `peers` will need to be updated to accommodate non-EVM peer addresses (e.g., `mapping(uint256=>bytes32)`). Alongside this, store the peer type per chain (e.g., an `isEvmPeer` flag or an enum) so the send path can dispatch the appropriate recipient validation. For EVM destinations, enforce canonicity before burning: in `bridgePositions` and `bridgeCollateral`.

Polymarket: Fixed in PR #306. `BridgeBase._validateRecipient()` now rejects any recipient with the upper 96 bits set and is applied on both `bridgePositions()` and `bridgeCollateral()`, so only canonical left-padded EVM addresses can be burned against. The receive path also compares the full 32-byte CCIP sender.

Cantina Managed: Verified.

3.1.2 OO-backed atomic neg-risk markets cannot truthfully settle the synthetic Other outcome

Severity: Medium Risk

Context: `OptimisticOracleModule.sol#L413`, `OracleAggregator.sol#L211`

Description: `OracleAggregator` configures atomic neg-risk requests with `resultLength==eventId.arity()`, meaning the result vector covers only the event's real outcomes. `NegRiskModule` separately derives the synthetic Other condition from leftover payout mass when all real outcomes resolve NO.

However, `OptimisticOracleModule` calls `getResultLength()` and treats that value as the full OO numerical outcome count. As a result, for an atomic neg-risk market with N real outcomes, the OO module accepts only prices `0..N-1`. Each accepted price resolves exactly one real outcome as YES and all other

real outcomes as NO. The valid Other answer, which would correspond to all real outcomes resolving NO, is rejected as invalid because index N is outside the accepted range. Therefore any OO-backed atomic neg-risk market whose true winner is Other cannot settle truthfully through the OO path. A truthful answer is rejected, while any accepted settlement necessarily marks one real outcome YES.

Recommendation: Make the OO encoding cover the full atomic neg-risk answer space. For an N-real-outcome atomic neg-risk request, accept numerical answers `0..N`, where `N` represents Other. The payout conversion should still return the aggregator's N-element real outcome vector. Prices `0..N-1` should produce a vector where the selected real outcome receives the full payout and every other real outcome receives zero. Price N, representing Other, should produce a vector where every real outcome receives zero, allowing `NegRiskModule` to derive the synthetic Other condition as YES.

Alternatively, explicitly forbid `OptimisticOracleModule` for atomic neg-risk markets unless the configuration proves the real outcomes are exhaustive and Other cannot be a valid winner.

Polymarket: Acknowledged. The synthetic Other is not meant to be resolved for atomic neg-risk markets. Neg-risk markets should have one of the real conditions always resolve to Yes. For atomic neg-risk, if every condition needs to be reported as NO, an empty condition should be renamed to Other, and UMA should resolve the real Other to YES, not the synthetic Other to YES.

Cantina Managed: Acknowledged.

3.2 Low Risk

3.2.1 Synthetic NegRisk fallback resolution bypasses per-condition pause

Severity: Low Risk

Context: `NegRiskModule.sol#L365-L374`

Description: `OracleModule` exposes `pauseResolution(ConditionId_id)` which stores a per-condition pause timestamp in `resolutionPausedAt[_id]`. The `onlyResolver` modifier enforces this pause for any condition directly submitted by a resolver.

NegRisk markets include a synthetic "Other" condition at index `conditionCount` whose payout equals `RESULT_DENOMINATOR-sum(allrealYESpayouts)`. This condition is never submitted directly — it is derived and stored automatically inside `_finalizeNegriskResolution` when either all real conditions are resolved or the aggregate YES payout reaches `RESULT_DENOMINATOR`.

The pause check in `onlyResolver` runs against the **submitted** condition ID. When the fallback is auto-derived, `_storeResult` is called on `fallbackConditionId` without reading `resolutionPausedAt[fallbackConditionId]`.

An admin who calls `pauseResolution(fallbackConditionId)` with the intent to block resolution of that condition will find the pause silently ignored when a resolver submits the triggering real condition.

The impact, however, is limited. The fallback payout is fully deterministic from already-accepted real-condition results, so a resolver cannot inject a manipulated value through this path. The admin retains the ability to block the fallback indirectly by pausing any unresolved real condition. However, the emergency control does not enforce its documented per-condition boundary for synthetic fallback resolution.

Recommendation: Before storing the fallback result, check whether resolution is paused for the fallback condition or document that it is not intended for this to be pauseable.

Polymarket: Fixed by redesign in PR #290. Resolution pause is now event-scoped — `resolutionPausedAt` is keyed by `EventId` and `onlyResolver` checks `resolutionPausedAt[id.eventId()]` (`OracleModule.sol:51,75`); the legacy migration path re-checks it too (`BaseMigrationMixin.sol:298`). Pausing an event necessarily blocks the synthetic fallback, since it can only fire from a real-condition report within the same (paused) event. The per-condition boundary flagged in the report no longer exists.

Cantina Managed: Verified.

3.2.2 Oracle Pause Consumes Dispute Window Liveness

Severity: Low Risk

Context: `OracleAggregator.sol#L301`, `OracleAggregator.sol#L315`

Description: When the `OracleAggregator` is paused via `pauseOracle()`, both `disputeResult()` and `reportResult()` are blocked by the `whenUnpaused` modifier. However, the dispute window is stored as an absolute timestamp (`disputeWindowEnd`) set at proposal time:

```
state.disputeWindowEnd = uint40(block.timestamp + cfg.livenessWindow);
```

Real time continues to advance while the oracle is paused. Neither `pauseOracle()` nor `unpauseOracle()` adjusts `disputeWindowEnd`. When the oracle is unpaused, the check in `disputeResult()` compares against wall clock time:

```
require(block.timestamp < state.disputeWindowEnd, DisputeWindowExpired());
```

If the pause duration is longer than the remaining dispute window, the window expires during the pause period and disputers have no opportunity to challenge the proposal. The outcome then becomes unconditionally finalizable by anyone via `finalize()`.

This gives an admin a mechanism to suppress disputes for a specific proposal: pause before the dispute window closes, hold the pause until it expires, then unpause. The dispute threshold is unreachable regardless of disputer intent. And even without malicious intent, emergency pauses triggered for unrelated reasons (e.g., a bug elsewhere in the system) silently erode liveness guarantees for any in-flight proposals and are only remediable if they act before someone calls `finalize`.

Recommendation: The dispute window should measure time during which disputes are *possible*, not wall-clock time. To enforce this, track cumulative paused duration in the `Pausable` mixin:

```
uint256 public totalPausedDuration;
uint256 private _pauseStartedAt;

function _pause() internal {
    _pauseStartedAt = block.timestamp;
    globalPaused = true;
}

function _unpause() internal {
    totalPausedDuration += block.timestamp - _pauseStartedAt;
    globalPaused = false;
}
```

Set `disputeWindowEnd` in adjusted time at proposal creation by offsetting by how much pausing has occurred:

```
state.disputeWindowEnd = uint40(block.timestamp - totalPausedDuration +
    ↪ cfg.livenessWindow);
```

And offsetting again with an updated pause duration when `disputeResult` is called:

```
require(block.timestamp - totalPausedDuration < state.disputeWindowEnd,
    ↪ DisputeWindowExpired());
```

Polymarket: Acknowledged. If we pause, it's most likely that we won't unpause the market again.

Cantina Managed: Acknowledged.

3.2.3 CcipBridge Cannot Interact With Non-EVM Peers

Severity: Low Risk

Context: CcipBridge.sol#L39

Description: The `CcipBridge` contract restricts the addresses of peer bridge contracts on other chains to the standard EVM 20-byte `address` type:

```
mapping(uint256 => address) public peers;
```

However, Chainlink CCIP is designed to support cross-chain communication across both EVM and non-EVM chain families (such as Solana, Cosmos, etc.). To accommodate non-EVM address formats which may not fit into 20 bytes, Chainlink CCIP defines the `receiver` field in `Client.EVM2AnyMessage` and the `sender` field in `Client.Any2EVMMessage` as `bytes`.

By hardcoding the `address` type for peers, `CcipBridge` throws away this protocol generality:

1. **Outgoing messages / Quotes:** In `_transportSend` and `_bridgeQuote`, the receiver is encoded via `abi.encode(peers[_dstChain])`. For a non-EVM destination chain, the peer address representation will be constrained to a 20-byte format, preventing valid configuration of non-EVM peer destinations.
2. **Incoming messages:** In `_ccipReceive`, the sender is decoded assuming a standard EVM `address`:

```
address sender = abi.decode(_message.sender, (address));
```

If a message is received from a non-EVM source chain, `_message.sender` (which contains the raw bytes representation of the sender on that chain) will fail to decode as an EVM `address` or decode to an incorrect address. This will revert the transaction or cause the `UnauthorizedSender` check to fail.

Recommendation: If supporting cross-chain communications with non-EVM chains is a requirement (now or in the future), update `CcipBridge` to store and compare raw peer bytes:

1. Update the `peers` mapping and the `PeerSet` event to use `bytes` instead of `address`:
2. Update `setPeer` to take `bytescalldata_peer` and validate that it is not empty:
3. Update `_transportSend` and `_bridgeQuote` to pass the stored peer bytes directly as the `receiver` without wrapping them in `abi.encode`.
4. Update `_ccipReceive` to perform a direct comparison of the raw sender bytes against the stored peer bytes (e.g., using `keccak256` hashing for dynamic bytes comparison).

If non-EVM support is strictly out of scope, document this limitation explicitly in the NatSpec comments for `peers` and `setPeer` so it is a known design constraint rather than a silent limitation.

Polymarket: Updated in PR #306. `peers` is now `mapping(uint256=>bytes32)` (`CcipBridge.sol:46`) — left-padded addresses for EVM chains, raw `bytes32` available for future non-EVM peers, no storage migration needed. The receive path decodes the CCIP sender as `bytes32` and compares all 32 bytes, which also rejects senders with dirty upper bits.

Cantina Managed: Partially Resolved. The recommendation above was implemented as stated. However, the fixes for issue 3.1.1 enforce that now all recipients are EVM addresses, so while the peer can be an address on a non-EVM chain, recipients cannot and therefore the bridge still cannot interact with non-EVM chains.

3.2.4 Callers can choose CCIP ordering policy for the shared bridge sender lane

Severity: Low Risk

Context: CcipBridge.sol#L168, CcipBridge.sol#L189

Description: `CcipBridge` exposes caller-supplied `_options` on public entrypoints and forwards them directly as CCIP `extraArgs`. This occurs through `BridgeRouter.bridgePositions()` / `bridgeCollateral()`, `BridgeBase.bridgePositions()` / `bridgeCollateral()` / `bridgeResult()`, and `CcipBridge._transportSend()`, which assigns `extraArgs:_options` without validation.

The gas-limit portion of `extraArgs` is largely a user-facing liveness footgun: a caller can provide insufficient destination gas, causing `ccipReceive()` execution to fail after the source-side burn. Chainlink's [manual execution](#) flow allows the gas limit to be overridden for recovery, so this is not necessarily permanent loss, but it can leave the transfer requiring manual intervention.

The more actionable issue is that `_options` also lets callers choose `allowOutOfOrderExecution`. Since all sends originate from the bridge contract, this is a shared CCIP sender lane, so user-selected ordered message can therefore affect ordering behavior for later bridge messages from other users. Chainlink's current EVM [best practices](#) state that in-order execution is being deprecated and that required lanes must set `allowOutOfOrderExecution` to true.

Recommendation: Do not forward arbitrary ordering policy from users. Decode or construct CCIP `extraArgs` in the bridge and enforce `allowOutOfOrderExecution=true` for all bridge messages. The gas limit can remain caller-configurable if that is the intended UX, but ordering should be protocol-controlled for the shared bridge lane. Also note that Chainlink states `extraArgs` [should be mutable](#) to allow for future compatibility.

Polymarket: Fixed in PR [#306](#). Callers can no longer pass raw CCIP `extraArgs`. `_options` must be empty (`DEFAULT_GAS_LIMIT=200k`) or `abi.encode(uint256gasLimit)`; anything else reverts `Invalid Options`. The bridge builds `GenericExtraArgsV2` internally with `allowOutOfOrderExecution` always true (`CcipBridge.sol:198–212`), so one stuck message can no longer block the shared sender lane.

Cantina Managed: Verified.

3.2.5 `bridgePositions()` does not enforce a source-side batch size cap

Severity: Low Risk

Context: [BridgeBase.sol#L180-L181](#)

Description: `bridgePositions()` accepts arbitrary-length `_positionIds` / `_amounts` arrays. `BridgeBase._sendPositionsMessage()` then builds a `BridgedPosition[]` for the full batch and encodes it through `BridgePayloads.positions()` without enforcing a maximum batch size or encoded payload length.

Since each `BridgedPosition` contains two 32-byte values, the encoded message data length is $97+64*n$ bytes. Chainlink's EVM CCIP [service limits](#) currently document a maximum message data length of 30 KB which caps the theoretical maximum at $n=467$.

This 30 KB payload limit is separate from destination execution gas. Payloads above the CCIP data limit should revert on the source chain in the router or fee quoter before the message is accepted, so they should not reach the DON or destination. In this contract, the source-side position burn occurs after `_sendPositionsMessage()`, so a source-side router revert should also prevent the burn.

However, payloads below 30 KB can still be too expensive to execute on the destination. `_handlePositionsReceive()` decodes the full array and calls `mintFromBridge()` once per position, so destination gas grows with batch size and receiver state. A much smaller batch than 467 can therefore be accepted by CCIP, burn source positions, and later require manual execution if the destination gas limit is insufficient.

Recommendation: Consider adding an explicit source-side cap for position bridge batches, ideally both by `positions.length` and encoded payload size. The cap should be well below the 30 KB theoretical maximum and based on destination gas tests for `_handlePositionsReceive()`. Apply the same validation in `quoteBridgePositions()` so oversized batches fail predictably before reaching the CCIP router.

Polymarket: Acknowledged. This is an extreme edge case and hard to put a limit for every single possible chain.

Cantina Managed: Acknowledged.

3.3 Gas Optimization

3.3.1 Event operations accumulate dead memory across outcome iterations

Severity: Gas Optimization

Context: [CombinatorialModule.sol#L506-L510](#), [CombinatorialModule.sol#L534-L538](#), [CombinatorialModule.sol#L584-L590](#)

Description: In Solidity, dynamic memory allocations are not reclaimed when a loop iteration ends or when an internal function returns. Instead, the free-memory pointer (located at `0x40`) continuously advances.

The functions `splitOnEvent`, `mergeOnEvent`, and `convertOnEvent` in `CombinatorialModule.sol` iterate over the outcomes of a `NegRisk` event. Within each loop iteration, they allocate memory to construct the child condition's leg array (via `_insertLeg`) and encode data to derive the condition's ID (via `getConditionId` calling `abi.encode`).

Once the child condition has been stored and its position minted or burned, these temporary arrays and encoding buffers are no longer needed. However, because the free-memory pointer is not reset, each subsequent loop iteration allocates memory above the previous iteration's dead buffers. Consequently:

- Peak memory usage grows linearly with the number of event outcomes multiplied by the size of the parent leg array.
- EVM memory expansion gas costs scale quadratically with the highest memory offset accessed. As a result, later iterations pay progressively higher gas costs due to the accumulation of dead memory.

Recommendation: Reset the free-memory pointer at the end of each outcome iteration in all three event operations. To do this safely, save a checkpoint of the free-memory pointer once after allocating any persistent memory-backed variables that must survive the loop, and restore the pointer to this checkpoint at the end of each iteration. For example, in `splitOnEvent`:

```
// >>> Checkpoint the free-memory pointer before the loop starts <<<
uint256 memoryCheckpoint;
assembly ("memory-safe") { memoryCheckpoint := mload(0x40) }

for (uint256 i; i <= conditionCount_; ++i) {
    ConditionId childConditionId =
        _storeLegsFromMemory(_insertLeg(parentLegs,
            ↪ _eventId.computeConditionId(i).computePositionId(0)));
    POSITION_MANAGER.mint(_to[i], childConditionId.computePositionId(0), _amount);

    // >>> Restore the pointer to reuse the checkpointed region <<<
    // Note: this is not "memory-safe"
    assembly { mstore(0x40, memoryCheckpoint) }
}
```

Polymarket: Fixed in PR #310.

Cantina Managed: Verified. `splitOnEvent()` / `mergeOnEvent()` / `convertOnEvent()` now snapshot the free-memory pointer before the outcome loop and reset it after each iteration.

3.4 Informational

3.4.1 Event-helper names overstate their NegRisk-only checks

Severity: Informational

Context: `CombinatorialModule.sol#L1030`, `CombinatorialModule.sol#L1041`

Description: `_requireEventNotPresent` and `_requireNoOtherEventLegs` appear to enforce module-agnostic event-level uniqueness. Both implementations enforce a narrower NegRisk-specific property:

```
if (leg.moduleId() == ModuleIds.NEGRISK && leg.conditionId().eventId() == _eventId) {
    revert EventAlreadyPresent();
}
```

Any leg whose module is not `NEGRISK` is skipped without comparing its derived event ID. `_requireNoOtherEventLegs` additionally skips the designated source index, but applies the same hardcoded module filter to every remaining leg. These helpers can therefore return successfully while a position conceptually associated with the event is present through another module. Their names do not communicate this module-specific behavior.

Recommendation: Rename both helpers to make their enforced properties explicit, for example: `_requireNegRiskEventNotPresent` and `_requireNoOtherNegRiskEventLegs`.

Polymarket: Acknowledged. It's fine.

Cantina Managed: Acknowledged.

3.4.2 `NegRiskModule` & `CombinatorialModule` Exceed EIP-170 24KB Limit

Severity: Informational

Context: `CombinatorialModule.sol#L1`, `NegRiskModule.sol#L1`

Description: The `NegRiskModule` and `CombinatorialModule` contracts fit under Polygon's PIP-30 runtime size limit of 32,768 bytes, but they exceed the standard EIP-170 limit of 24,576 bytes. EIP-170 is strictly enforced on Ethereum Mainnet and major L2 networks targeted for cross-chain deployments including Base, BSC, Optimism, and Arbitrum.

The current runtime sizes compiled by Foundry and validated via the `makesizes` script show:

- `NegRiskModule` : Compiled size is 28,167 bytes (exceeding the EIP-170 limit by 3,591 bytes).
- `CombinatorialModule` : Compiled size is 27,865 bytes (exceeding the EIP-170 limit by 3,289 bytes).

Impact on Cross-Chain Deployments: Today, the build CI only validates against the 32,768-byte Polygon limit, allowing the contracts to pass check-size checks. However, neither contract can be deployed to any of the five target 24KB EVM networks.

This directly impacts `CcipBridge` integration:

1. `CcipBridge` relies on `BridgeBase` (`src/bridge/abstract/BridgeBase.sol`), where `mintFromBridge` and `burnFromBridge` invoke operations on the modules corresponding to the bridged positions.
2. In the target destination chains, `PositionManager` validates that caller modules are registered in `moduleById[moduleId]`.
3. If `NegRiskModule` or `CombinatorialModule` cannot be deployed on these target destination chains due to EIP-170 size violations, bridging multi-outcome (neg-risk) and combinatorial positions to and from these chains will be completely blocked.

Recommendation:

1. Refactor and Reduce Contract Sizes Below 24,576 Bytes: To make the modules deployable on standard EVM networks, apply size reduction strategies:

- **Separate Migration Logic:** Legacy CTF migration functionality (e.g., in `NegRiskMigrationMixin` / `BinaryMigrationMixin`) accounts for significant bytecode overhead. Extract migration functions into a standalone, temporary helper contract that handles migration transfers and interacts with the main module externally.
- **Externalize Helpers to Libraries:** Move complex helper methods (e.g., validation logic, combinatorial path verification, outcome calculations) into separate external libraries or delegatecall execution helpers.
- **Consolidate Redundant Logic:** Review and deduplicate logic patterns, ensuring common checks are unified.

2. Extend size checks to enforce EIP-170 by default in CI: Modify `bash/check-contract-sizes.sh` to enforce the standard 24,576B limit globally, while allowing specific exceptions for Polygon-only contracts.

3. Update Documentation: Add references to the per-chain size limitations in `CLAUDE.md` and the deployment/CI documents once the refactoring path is decided.

Polymarket: Acknowledged. Aware of this. We will remove the `MigrationMixin` when deploying to other chains to bring size below 24KB.

Cantina Managed: Acknowledged.

3.4.3 Peer rotation requires a bridge-wide send pause

Severity: Informational

Context: [BridgeBase.sol#L146-L155](#)

Description: `BridgeBase` exposes only a global outbound pause. Safely rotating one destination peer requires stopping new messages to that destination and waiting for its in-flight messages to settle before changing the destination's peer configuration. On a bridge serving multiple destinations, the current control therefore requires halting every outbound lane to perform maintenance on one lane.

The global pause can prevent messages from being stranded during a coordinated rotation, so this is not independently a loss-of-funds issue. However, it unnecessarily disrupts unrelated routes and may discourage operators from draining a lane before changing its peer.

Recommendation: Add a per-destination outbound pause while retaining the global emergency pause. Before rotating a peer, pause sends to that chain and use the CCIP message IDs to confirm that all pending transfers have completed on the destination.

Polymarket: Fixed in PR [#306](#). Per-lane `chainSendPaused` / `chainReceivePaused` mappings with admin-gated pause/unpause per chain, layered on the global flags (`BridgeBase.sol:41-44` , enforced at L180/L187 with distinct `ChainSendIsPaused` / `ChainReceiveIsPaused` errors). One lane can now be drained and its peer rotated without halting other routes; `removePeer` remains the owner-gated permanent kill switch.

Cantina Managed: Verified.

3.4.4 Incorrect BridgeBase storage gap size shifts derived peers storage slot in CcipBridge

Severity: Informational

Context: CcipBridge.sol#L39

Description: BridgeBase already declares two storage slots before its storage gap. With a 49-slot gap, the base contract reserves 51 slots total, so the first storage variable in a derived contract, such as the peers mapping in CcipBridge, starts at slot 51. If the intended layout is for peers to begin at slot 50, the gap is one slot too large.

Recommendation: Reduce the storage gap in BridgeBase from 49 slots to 48 slots:

```
uint256[48] private __gap;
```

This preserves the two existing slots and causes derived storage to begin at slot 50 as intended.

Polymarket: Resolved in PR #306 alongside the per-lane pause storage. BridgeBase now declares 4 slots + uint256[46]__gap (BridgeBase.sol:47), so the base reserves exactly 50 slots and CcipBridge.peers begins at slot 50 as intended. Verified by the StorageSlots suite and updated storage-layouts baselines; CcipBridge had no live deployment, so the slot shift is safe.

Cantina Managed: Verified.